

accenture

TAP Steuerungsgesellschaft powered by Thüga

AYT TAP Rollout – Jira & Confluence Integration

JENA 6/2/2025

Liam Thomson

Comments

This presentation covers the Ask Your Template platform in the context of the TAP Rollout Team and focuses on my work extending it with Jira and Confluence. I will explain what AYT is, why it was chosen, how the integrations work, and where the current limitations are. The implementation should be understood as a proof of concept, not a finished product. I will also outline remaining TODOs and trade-offs inherent to RAG systems. The goal is to show technical feasibility and architectural soundness rather than a production-ready system.

TAP

Copyright © 2024 Accenture. All rights reserved.

What is AYT?

- Retrieval-Augmented Generation (RAG) pipeline
- Ingests & vectorizes unstructured knowledge
- Enables natural-language querying
- Integrated in 4 projects so far
- Swiss Based Project

Supported Sources

- Local files
- SharePoint
- ****
- M2C Portal
- **New: Jira & Confluence**

TAP

Copyright © 2024 Accenture. All rights reserved.

Comments

AYT is a Retrieval-Augmented Generation pipeline that combines vector search with large language models. Instead of training a model on internal knowledge, it retrieves relevant content from a vector database and uses it as context for answering questions. This makes it flexible and easier to maintain. At the time of this work, AYT already supported several internal sources, but Jira and Confluence were missing. Those two systems are central knowledge hubs for the Rollout Team, which made them the focus of this implementation.

Vector database:

Douze, M., Guzhva, A., Deng, C., Johnson, J., Szilvasy, G., Mazaré, P.-E., Lomeli, M., Hosseini, L., & Jégou, H. (2024). The Faiss library. arXiv. https://arxiv.org/abs/2401.08281

RAG:

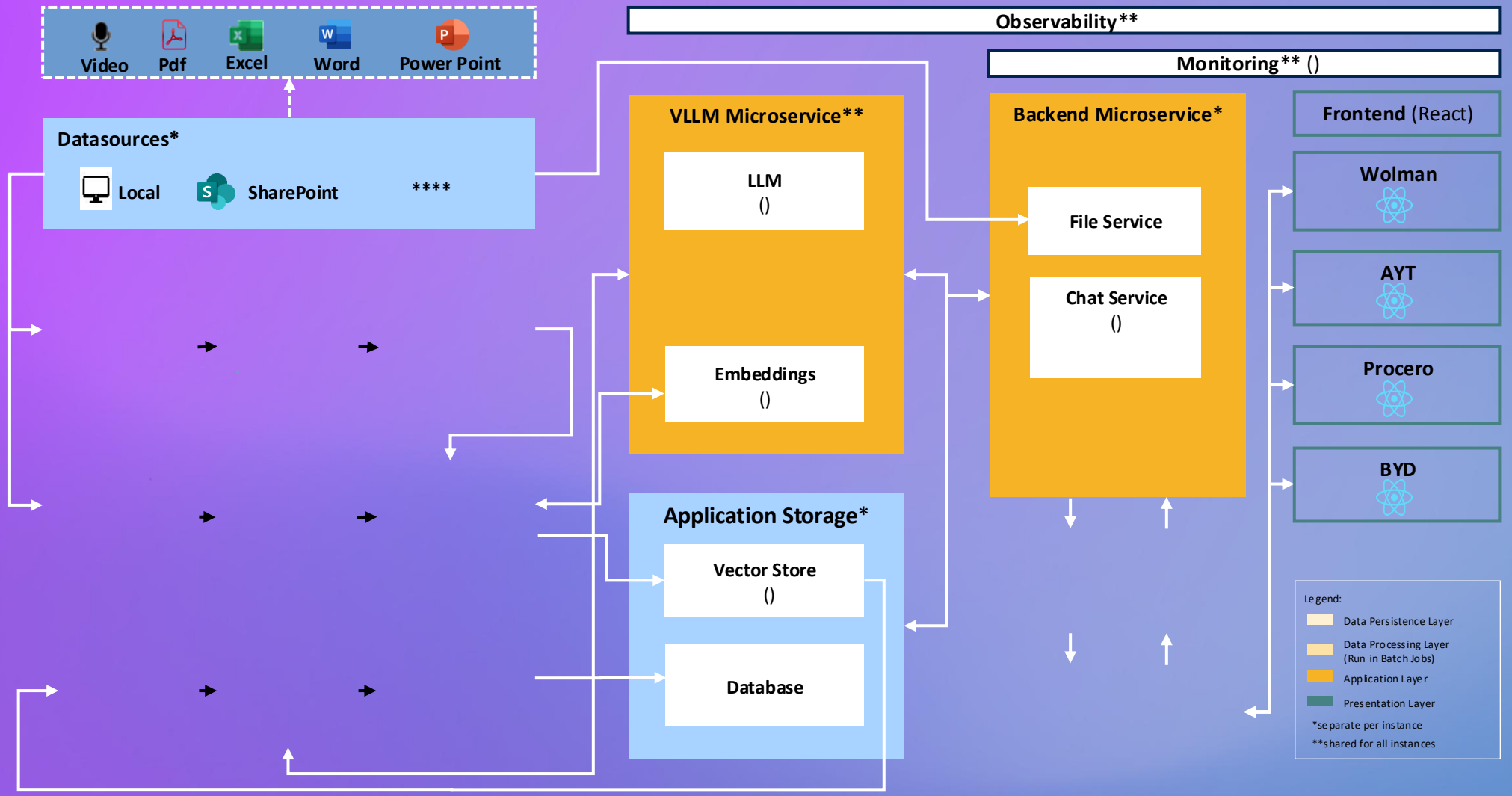
Gupta, S., Ranjan, R., & Singh, S. N. (2024). A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions. arXiv. https://arxiv.org/abs/2410.12837

TAP

Copyright © 2024 Accenture. All rights reserved.

Ask Your Template: **Architecture (M2C Portal)**

An open-source architecture that can be deployed completely in-house



TAP

Comments

On this slide, I'll briefly walk through the architecture diagram. Parts of it have been redacted here due to privacy constraints, but the full version was shown during the actual presentation. The diagram illustrates how the different services interact with each other: data sources such as Jira and Confluence feed into the ingestion layer, where content is processed, chunked, and vectorized, and stored in the vector database. User queries are handled by the chat service, which retrieves relevant context from the vector database and passes it to the LLM for answer generation. The key takeaway from this diagram is the clear separation of concerns between ingestion, storage, retrieval, and generation, which makes the system modular and extensible.

Motivation & Problem Statement

Problem

- Knowledge spread across multiple silos
- Jira, Confluence, documents, portals
- Manual search is slow and error-prone

Need

- Unified, intelligent search
- One interface for internal & client questions

TAP

Copyright © 2024 Accenture. All rights reserved.

Comments

Within the TAP Rollout Team, relevant knowledge is distributed across many systems. Important information might live in Jira issues, Confluence pages, or attached documents. Finding answers often requires manual searching and context switching. This becomes even more problematic when answering recurring client questions. The motivation was therefore to reduce friction and centralize knowledge access through a single intelligent query interface.

TAP

Copyright © 2024 Accenture. All rights reserved.

Why AYT?

Why AYT was chosen

- Already integrated with M2C Portal
- Modular source architecture
- RAG instead of model training
- Actively developed platform

Gap identified

- Missing Jira & Confluence integration

TAP

Copyright © 2024 Accenture. All rights reserved.

Comments

During the conceptual phase, AYT was identified as an existing solution already used within the organization. It closely matched the Rollout Team’s requirements and avoided reinventing the wheel. Its modular architecture made adding new sources feasible without rewriting the system. The main functional gap was the lack of Jira and Confluence integration, which naturally became the focus of my work.

TAP

Copyright © 2024 Accenture. All rights reserved.

Local Setup & AI Refinery Chat Integration

Local setup

- Backend + Accenture-themed frontend
- Provided by AYT product owner

LLM integration

- AI Refinery chat endpoints
- Used for answering queries

TAP

Copyright © 2024 Accenture. All rights reserved.

Comments

I received a minimal backend build and a frontend styled according to Accenture branding. This setup was deployed locally to allow rapid iteration. For the LLM layer, the system was connected to AI Refinery chat endpoints. These are used to generate natural-language answers based on retrieved context. In local testing, the chat functionality worked reliably and formed the basis for all query responses.

AI Refinery is Accentures internal AI platform that gives access to various models on servers, for LLM, Embedding, Transcription, and more.

TAP

Copyright © 2024 Accenture. All rights reserved.

AI Refinery Embeddings: Attempt & Limitation

Embedding attempt
- AI Refinery embedding models

Observed behavior
- Works for small Jira test sets (3–4 issues)
- Fails at scale (~1800 issues)

Result
- HTTP 500 / 501 errors
- Switched to local embedding model

TAP

Copyright © 2024 Accenture. All rights reserved.

Comments

I also attempted to use AI Refinery embedding models for vector generation. While this worked for very small test cases, larger ingestion runs consistently failed with server-side errors. This made the approach unsuitable for real Jira projects with thousands of issues. As a result, embeddings were generated using a local model instead. This highlights practical scalability considerations when integrating managed services.

TAP

Copyright © 2024 Accenture. All rights reserved.

Source Architecture & Processing Model

Architecture

- Lazy-loading sources
- Metadata first, content on demand

Processing split

- Vectorized content → semantic search
- Metadata → filtering & traceability

Key concept

- Chunking is source-specific

TAP

Copyright © 2024 Accenture. All rights reserved.

Comments

AYT uses a lazy-loading architecture where sources first enumerate lightweight document stubs containing only metadata. Full content is fetched only when needed for vectorization or querying. Each document is split into vectorized text and metadata. Chunking is a critical configuration point and must be adapted per source, since Jira issues and Confluence pages have very different text structures. Poor chunking directly impacts retrieval quality.

Lazy loading:

Wikipedia Contributors. (2025, November 28). Lazy loading. Wikipedia. https://en.wikipedia.org/wiki/Lazy_loading

Vectorization:

Wikipedia Contributors. (2026, January 29). Vectorization (mathematics). Wikipedia. https://en.wikipedia.org/wiki/Vectorization_%28mathematics%29

Vectorization:

Wikipedia Contributors. (2026, January 29). Vectorization (mathematics). Wikipedia. https://en.wikipedia.org/wiki/Vectorization_%28mathematics%29

Chunking in RAG:

Dev Nerd. (2025, May 21). Understanding chunking and vectorization in retrieval-augmented generation (RAG). Medium. https://medium.com/@dev.nerd87/understanding-chunking-and-vectorization-in-retrieval-augmented-generation-rag-172cbb46bd79

TAP

Copyright © 2024 Accenture. All rights reserved.

Jira Source Integration

Jira as AYT Data Source

- First-class AYT source, enumerated via **JQL**
- Lazy loading of issue content (fetched only when needed)

Vectorized Content

- Issue title (key + summary)
- Description
- Comments

Chunking Strategy

- Ignore sections < 50 chars (noise filtering)
- Skip trivial issues entirely
- Chunk size: **300–1200 chars**
- Title prefixed to every chunk

Metadata

- Strict allow-list (status, priority, assignee, escalation, ownership, timestamps)

TODOs

- Broader testing on real TAP projects
- Metadata validation with Rollout Team
- Performance testing at scale
- Fine-tuning chunking thresholds

Copyright © 2024 Accenture. All rights reserved.

Comments

The Jira source was implemented as a fully integrated AYT data source and is the most complete part of this work so far. Issues are discovered via JQL and loaded lazily, which means content is only fetched when required for vectorization or querying. Only text-rich fields are vectorized, namely the issue title, description, and comments, to avoid embedding administrative noise. A custom chunking strategy was implemented that filters out trivial sections, targets 300 to 1200 characters per chunk, and always prefixes the issue title to preserve context. Metadata is strictly allow-listed to prevent schema growth and is used for filtering and traceability. While the implementation is functional, it still needs broader testing, performance validation, and some fine-tuning before being considered production-ready.

TAP

Copyright © 2024 Accenture. All rights reserved.

Confluence Source Integration

Confluence as AYT Data Source

- Implemented as **Proof of Concept**
- Pages enumerated via **CQL**
- Lazy loading of page bodies

Processing

- Page body fetched as **storage HTML**
- Lightweight HTML → plain-text conversion
- Structure preserved via line breaks

Current Limitation

- One plain-text blob per page
- No chunking (not scalable)

Planned Improvements

- Chunking: **300–1200 chars**
- Prefix chunks with page title
- Skip trivial sections
- Add chunk-level testing

TAP

Copyright © 2024 Accenture. All rights reserved.

Comments

The Confluence integration was intentionally implemented as a proof of concept to validate feasibility and architectural fit. Pages are discovered using CQL queries and fetched lazily, similar to Jira. The page body is retrieved in storage HTML format and converted into plain text using a lightweight approach that preserves basic structure without introducing a full HTML parser. At the moment, each page is returned as a single text blob, which is sufficient for demonstration but not suitable for scalable retrieval. The next logical step is to introduce a chunking strategy similar to Jira, which will significantly improve retrieval quality and make the Confluence source production-ready.

TAP

Copyright © 2024 Accenture. All rights reserved.

RAG Limitations & Trade-offs

Limitations

- LLMs can hallucinate
- Retrieval quality depends on data

Why RAG anyway

- No model training required
- Knowledge stays up to date
- Scales better than rule-based bots

TAP

Copyright © 2024 Accenture. All rights reserved.

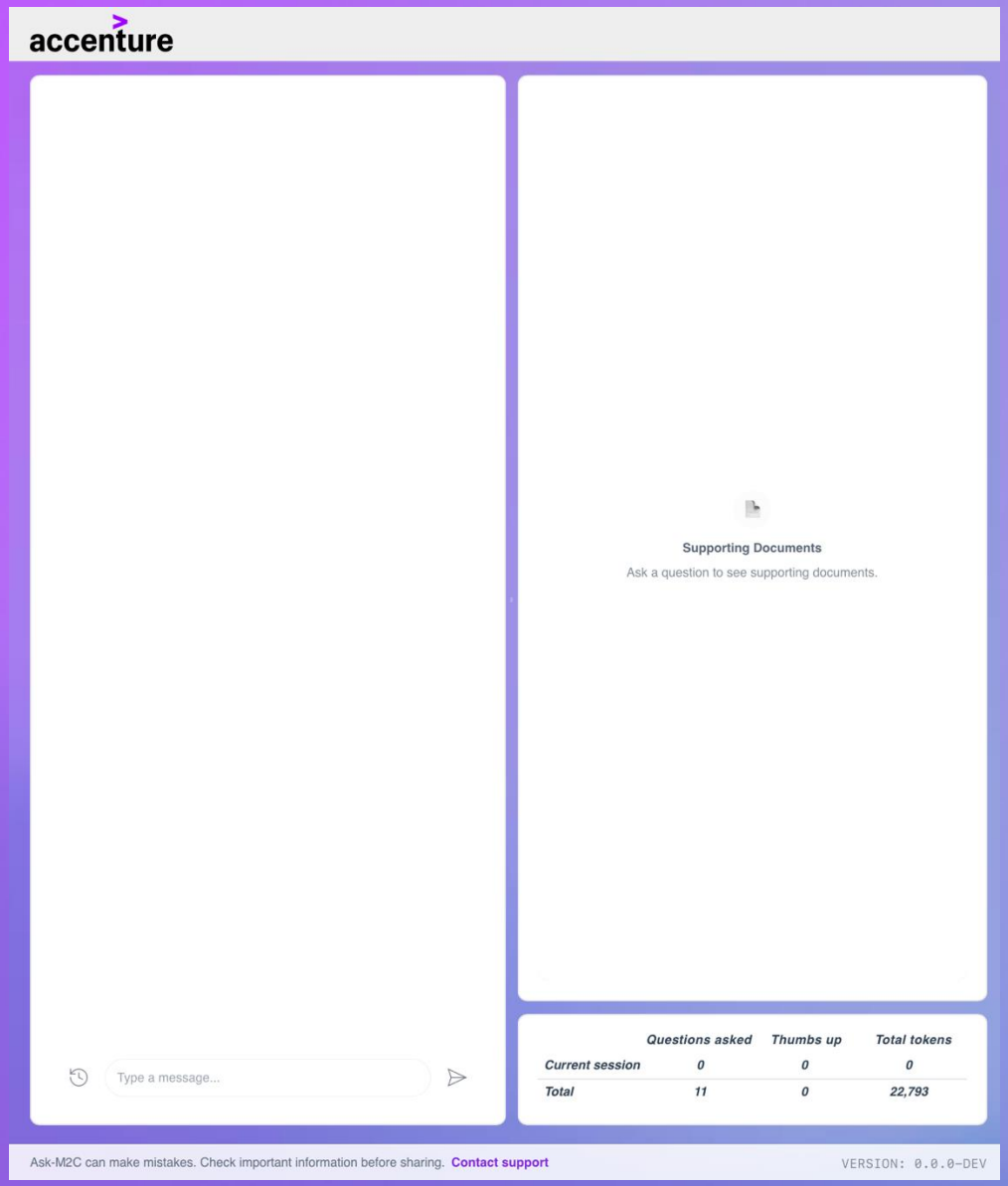
Coments

RAG pipelines do not eliminate hallucinations, and answer quality is bounded by the quality of ingested content. However, compared to training models from scratch, fine-tuning static datasets, or maintaining rule-based chatbots, RAG is the most pragmatic solution. It supports evolving knowledge, avoids locking information into model weights, and preserves traceability to source systems.

TAP

Copyright © 2024 Accenture. All rights reserved.

Demo




```python
class JiraSource(DocumentSource):
    """Jira source that exposes issues as virtual text documents.

    High-level flow:
    1) `enumerate_documents()` pages through Jira search results (lightweight metadata only).
    2) For each issue, we create a `SourceDocument` stub with a lazy `content_loader`.
    3) When content is actually needed, we fetch the full issue (summary/description) and
       optionally comments, render them to plain text, then chunk to 300-1200 characters.

    Why plain text:
    - The downstream vector store expects a text payload for embedding.
    - We keep structured fields in metadata and only embed the human-readable content.

    Vectorized content fields (embedded):
    - summary (included in the title line)
    - description
    - comments
```

TAP

Copyright © 2024 Accenture. All rights reserved.

Comments

At this point I give a live demo of the local instance. Due to privacy contraints, I can only show a static image of the ui.

I also delve into the source code and show some of the files, like: config.yaml, jira_source.py, source_factory.py and so on.

TAP

Copyright © 2024 Accenture. All rights reserved.

Conclusion & Next Steps

Summary

- Jira integrated & chunked
- Confluence PoC completed
- AI Refinery chat integrated

Next steps

- Testing & hardening
- Confluence chunking
- Deployment considerations

TAP

Copyright © 2024 Accenture. All rights reserved.

Comments

In summary, Jira and Confluence were successfully integrated into AYT, with Jira reaching an advanced implementation stage and Confluence serving as a proof of concept. The system demonstrates that a unified knowledge query interface for the Rollout Team is technically feasible. The next steps focus on testing, refinement, and preparing the system for a potential production deployment.

TAP

Copyright © 2024 Accenture. All rights reserved.



TAP

Copyright © 2024 Accenture. All rights reserved.